

プロジェクト概要: AkiNavi HR-AI (アキナビ HR-AI)

1. ビジョン

「案件の空き」と「人材 (HR) リソース」をAIで最適に結びつける、高精度・高品質なマッチング基盤。バグゼロを目指した徹底的なテスト、こまめなGit管理、および将来の担当者が即座に再現可能なドキュメント完備をゴールとする。

2. 技術スタック

- **Frontend:** React 19 (Vite 8), TypeScript, Tailwind CSS v4, TanStack Query v5
- **Backend/DB:** Supabase (PostgreSQL, Edge Functions, Realtime, pg_cron, pg_net)
- **AI (ブラウザ) :** Google Gemini デフォルト `gemini-2.5-flash-lite` (`VITE_GEMINI_MODEL` で上書き可)。人材・案件登録の UI からは Gemini を使わなくなった (コミット `f28ec86` で「AI で登録」ボタン廃止)。`src/lib/ai/geminiProvider.ts` は残るが UI 接続なし
- **ファイルパース (ブラウザ) :** `xlsx` (Excel)・`mammoth` (Word) — `src/lib/fileParser.ts`。PDF と画像は現状未対応 (旧 `pdfjs-dist` は依存に残るが import なし。CandidatePage / ProjectPage で「PDF は手動貼り付けか画像化して添付」を案内)
- **AI (サーバー・メール解析) :** AI 不使用。`inbound-email` Edge Function は regex (`extractCandidateFieldsRegex`) + 文章スキャン (`extractFromProse`) + `skill_master` DB 照合 + 駅→都道府県マッピング のみで構造化抽出する (コミット `139a4f2` で AI 廃止、`a4dc3b4` で `classifyInboundRelevance` / `generateJSONSmart` 等のデッドコードも完全削除済み)。残存 AI 呼び出しは自動マッチング用 `matchCandidateToProject` (`AUTO_MATCH_ENABLED='true'` 時の即時マッチ) のみ
- **AI (サーバー・マッチング・新方式 `match-batch`) :** コミット `b35df40` で導入。ルールベース事前フィルタ (スキル一致40pt / 経験15pt / 単価15pt / 勤務地20pt / リモート10pt = 100pt) → topN だけバッチ AI 採点。AI フォールバック順は Cerebras `llama3.1-8b` → Groq `llama-3.3-70b-versatile` → Gemini `gemini-2.5-flash`。AI が 3 段とも失敗したらルールスコアで全代替 (`usedModel='rule'`)。1 案件 = 1 AI コール (topN 候補者を一括採点) でトークン消費を圧縮
- **AI (サーバー・マッチング・既存 `match-score`) :** UI 手動の単発スコア計算 (`duplicateSuspected` フラグ込み)。フォールバック順は同上。マッチング理由は **150 字以内**で「必須スキル合致 → 経験年数 → 単価 → 勤務地リモート → 懸念点」の優先順
- **AI (サーバー・マッチング・自動 cron `auto-match`) :** 毎朝 JST 9:00 起動。`match-batch` を経由して Cerebras/Groq/Gemini フォールバック付き。`MAX_CANDIDATES_PER_PROJECT=40 / BATCH_AI_SIZE=20`、`app_config.auto_match_enabled='false'` でスキップ
- **AI (サーバー・メール種別分類) :** `poll-email` 内で Gemini バッチ分類 (任意・既定 `app_config.email_classify_enabled='false'`)
- **AI (切替・フロントのみ) :** `VITE_AI_PROVIDER=gemini / openai` — OpenAI は未実装スタブ
- **メール自動取り込み (現行・稼働中) :** Microsoft Graph API ポーリング + Supabase pg_cron (Make.com不要・完全無料・5分間隔)
- **メール自動取り込み (旧・現在停止中) :** Make.com → Pipedream (いずれも無料枠超過により運用停止)
- **Edge Function デプロイ事前検査:** `scripts/check-and-deploy-edge.sh` (`a8422fb`)。deno `check` で TS2304 系 (未定義変数) を検知し、見つければ deploy 中止
- **Testing:** Vitest, React Testing Library, MSW (Mock Service Worker)。`scripts/verify_email_extraction.mjs` でメール抽出口ジックのリグレッション検証
- **Deployment:** Vercel (Frontend), Supabase (Backend)

3. 開発・品質管理工程表（人間とClaude Codeの共同作業）

【Phase 0】リポジトリ準備・初期化・インフラ開通

1. **【人間】手作業:** GitHubでPrivateリポジトリ作成、Vercelインポート、APIキー取得
2. **【Claude】作業:** Vite + React + TypeScriptプロジェクト初期化、主要ライブラリインストール、Git初期化・push

【Phase 1】DB基盤と環境構築

1. **【Claude】作業:** jsonbとGINインデックスを用いた汎用DB設計(SQL)の提示
2. **【人間】手作業:** SupabaseでSQL実行、.env / Vercel環境変数の設定
3. **【Claude】作業:** 接続確認後、DBスキーマ定義をcommit & push

【Phase 2】コアロジック開発 & 単体テスト

1. **【Claude】作業:** AI解析エンジン（Edge Functions）およびスコアリングロジックの実装
2. **【Claude】開発:** 単体テスト（Unit Test）の記述
3. **【Claude】作業:** テスト通過を確認し、commit & push

【Phase 3】UI実装 & 結合テスト

1. **【Claude】作業:** ランキング表示、マッチング画面、提案履歴機能の実装
2. **【Claude】開発:** 結合テスト（Integration Test）の記述
3. **【人間】手作業:** 実際のメールデータを用いた解析・マッチング精度の最終検証
4. **【Claude】作業:** 完了後、commit & push

【Phase 4】Make.com 連携・UI改善・デモ環境整備 （完了・メール連携は停止中）

1. **【Claude】作業:** Make.com (Outlook) と連携した自動解析フローの実装
 - フロー: メール受信 → Make.com が検知 → Edge Function `inbound-email` → Gemini AI 解析 → DB 保存
 - 人材用メール: `akinavi.hr.ai.voice.human@outlook.jp`
 - 案件用メール: `akinavi.hr.ai.voice.project@outlook.jp`
2. **【Claude】作業:** AI解析精度の継続的改善（プロンプトチューニング）
3. **【Claude】作業:** Google Drive / Sheets / Docs リンクの自動取得機能実装
4. **【Claude】作業:** デモ環境（data_env）分離・DemoSeedPanel実装
5. **【Claude】作業:** 人材・案件の編集機能・最終更新者/日時表示
6. **【人間】判断:** Make.com・Pipedreamともに無料枠超過 → Microsoft Graph API ポーリングへ移行決定

【Phase 4.5】Microsoft Graph API ポーリングへ移行 （完了・稼働中）

方針

Make.com・Pipedream等の外部SaaSを廃止し、**完全無料・永続稼働**のメール自動取り込みを実現する。

新フロー:

Outlook受信（5分以内）

↓

```
Supabase pg_cron (5分ごとに起動)
↓
Edge Function: poll-email
- Graph APIでアクセストークン取得 (リフレッシュトークンから)
- GET /me/messages (未読メールを取得)
- 既存の inbound-email と同じ解析ロジックへ渡す
- 処理済みメールを既読にマーク
↓
Gemini AI 解析 → DB 保存
```

コスト試算 (すべて無料枠内) :

コンポーネント	無料枠	予想消費量 (5分に1回)
pg_cron	制限なし	約 8,640回/月
pg_net	制限なし	約 8,640回/月
Edge Functions	500,000回/月	約 8,640回/月
Microsoft Graph API	無料	約 8,640回/月

作業一覧

【人間】手作業 (Claude Codeでは実施不可)

1. Azureアプリ登録 (所要: 約30分)

- <https://portal.azure.com> にアクセス (Microsoftアカウントでログイン・無料)
- 「Microsoft Entra ID」 → 「アプリの登録」 → 「新規登録」
- 名前: 任意 (例: **akinavi-mail-poller**)
- サポートされるアカウントの種類: 「個人用Microsoftアカウントのみ」を選択
- リダイレクトURL: **<http://localhost>** を追加
- 登録後、以下を控える:
 - **クライアントID (アプリケーションID)**
 - 「証明書とシークレット」 → 「新しいクライアントシークレット」 → **クライアントシークレット (値)**
- 「APIのアクセス許可」 → 「アクセス許可の追加」 → Microsoft Graph → 委任されたアクセス許可
 - **Mail.Read** を追加
 - **Mail.ReadWrite** (既読マーク用) を追加

2. リフレッシュトークン取得 (2アカウント分) (所要: 約30分)

- 以下のURLをブラウザで開き、**human@outlook.jp** でログイン:

```
https://login.microsoftonline.com/consumers/oauth2/v2.0/authorize
?client_id=<クライアントID>
&response_type=code
&redirect_uri=http://localhost
&scope=offline_access Mail.Read Mail.ReadWrite
```

- ログイン後にリダイレクトされるURL (<http://localhost/?code=...>) から `code=` の値を控える
- Claude Codeに渡してリフレッシュトークンに交換 (次のClaude作業で実施)
- **project@outlook.jp** でも同様に繰り返す

3. Supabase Secrets に登録 (所要: 約10分)

- Supabase Dashboard → Project Settings → Edge Functions → Secrets
- 以下を追加:

シークレット名	値
GRAPH_CLIENT_ID	AzureのクライアントID
GRAPH_CLIENT_SECRET	Azureのクライアントシークレット
GRAPH_REFRESH_TOKEN_HUMAN	human@outlook.jp のリフレッシュトークン
GRAPH_REFRESH_TOKEN_PROJECT	project@outlook.jp のリフレッシュトークン

4. SupabaseでSQL実行 (所要: 約5分)

- [supabase/migrations/add_email_polling_cron.sql](#) をSQL Editorで実行
- pg_cron ・ pg_net の有効化とスケジュール登録

【Claude Code】作業 (完了)

1. Edge Function **poll-email** の実装

- [supabase/functions/poll-email/index.ts](#) を新規作成
- 4アカウント (human/prod, project/prod, human/demo, project/demo) を [Promise.allSettled](#) で並列処理
- リフレッシュトークンを [app_config](#) テーブルでローテーション保存 (フォールバック: Secrets)
- [resolveCallKey\(\)](#): [INBOUND_CALL_KEY](#) → [SUPABASE_SERVICE_ROLE_KEY](#) → [SUPABASE_ANON_KEY](#) の順で [eyJ](#) 始まりJWTを使用
- 処理失敗時は [markAsUnread](#) で元に戻し次回再試行

2. pg_cronマイグレーションSQL作成

- [supabase/migrations/add_email_polling_cron.sql](#) を作成
- pg_cron ・ pg_net の有効化、[app_config](#) の UNIQUE 制約追加
- 5分ごとに [poll-email](#) を起動するスケジュール登録

3. 動作確認: 4アカウント全て正常稼働 (未読メール → 解析 → DB保存 → 既読マーク)

【Phase 4.6】Microsoft OAuth UI連携機能の追加 (完了)

Supabase Secrets への手動リフレッシュトークン登録を廃止し、設定画面からワンクリックで Microsoft アカウントを連携できる OAuth フローを実装した。

フロー

設定画面で「連携する」をクリック

↓

Edge Function microsoft-oauth (step=start) → authorize URL 生成

↓

ブラウザが Microsoft ログインページへリダイレクト

↓

ユーザーがログイン・権限承認

↓

Microsoft が <origin>/auth/callback?code=...&state=<account> にリダイレクト

↓

AuthCallbackPage が code を受け取り Edge Function microsoft-oauth (step=callback) を呼び出す

↓

Edge Function が code → refresh_token を交換し app_config に保存

↓

成功メッセージ表示・設定ページへ誘導

追加・変更ファイル

ファイル	変更内容
supabase/functions/microsoft-oauth/index.ts	新規作成：authorize URL 生成 & code 交換 Edge Function
src/pages/AuthCallbackPage.tsx	新規作成：OAuth コールバック専用ページ
src/pages/SettingsPage.tsx	変更：Microsoft アカウント連携セクション追加
src/App.tsx	変更： <code>/auth/callback</code> パスで AuthCallbackPage を表示
src/lib/db/emailSettings.ts	変更： <code>getConnectionStatuses()</code> 関数追加

app_config キー

キー	内容
graph_rt_human_prod	human@outlook.jp (prod) のリフレッシュトークン
graph_rt_project_prod	project@outlook.jp (prod) のリフレッシュトークン
graph_rt_human_dev	human dev (demo) のリフレッシュトークン
graph_rt_project_dev	project dev (demo) のリフレッシュトークン
graph_connected_<account>	連携済みフラグ (" <code>true</code> " / 未設定)

Azure アプリへの追加設定（手動）

- Azure ポータル → アプリ登録 → 認証 → リダイレクト URI に `<origin>/auth/callback` を追加すること

【Phase 4.7】Box連携 （実装完了・人間手作業待ち）

BoxはOAuth2なしで機械的なファイル取得ができないため、古いWindows PCをデータ取得専用機とした**box-downloader**バッチを別プロジェクトで作成し、Googleスプレッドシートをキューとして本アプリと連携する設計。

全体フロー

① メール受信 (inbound-email 改修済み)

– メール本文中の Box URL (app.box.com/s/xxx) を検出

– candidates.box_url に保存、box_status = 'pending' で人材を仮登録

– Googleスプレッドシートの boxurl 列に書き込み (キュー登録)

② box-downloader バッチ (別プロジェクト・Windows PC・1日1回手動 or タスクスケジューラ)

– スプレッドシートの「実施=空欄」行を読み込む

– Playwright (Chromium) で Box 共有URLへアクセスしファイルをダウンロード

– Google Drive の指定フォルダへアップロード

– スプレッドシートの driveurl 列・実施列 (成功/失敗) を更新

③ enrich-candidate バッチ (Supabase Edge Function・毎日 JST 3:00 自動実行)

※ box-downloader の実行 (②) が完了した後に走るよう時刻設定

– Googleスプレッドシートを読み込む (実施=成功 & driveurl あり)

– candidates テーブルで box_url が一致 & box_status='pending' の人材を検索

– Drive URL からファイルを取得 (fetchGoogleLinks の既存ロジックを流用)

– Gemini で再解析 → 既存レコードを UPDATE

– box_status = 'enriched' に更新

スプレッドシート構造 (box-downloaderと共有キュー)

列	名前	書き込み元	内容
A	boxurl	inbound-email	Box共有URL
B	driveurl	box-downloader	Google Drive URL (アップロード後)
C	実施	box-downloader	空欄 / 成功 / 失敗

DB変更 (candidatesテーブル)

カラム	型	内容
box_url	text	メールから抽出したBox共有URL
box_status	text	pending (未処理) / enriched (更新済み) / failed (失敗)

新規ファイル一覧

ファイル	内容
supabase/migrations/add_box_columns.sql	candidates に box_url, box_status 追加

ファイル	内容
<code>supabase/functions/enrich-candidate/index.ts</code>	スプレッドシート読み込み → Drive取得 → 再解析 → UPDATE
<code>supabase/migrations/add_enrich_cron.sql</code>	enrich-candidate を毎日 JST 3:00 に起動する pg_cron
<code>../box-downloader/</code>	別プロジェクト (Node.js/TypeScript/Playwright)

改修ファイル一覧

ファイル	変更内容
<code>supabase/functions/inbound-email/index.ts</code>	Box URL 検出 → スプレッドシート書き込み + candidates.box_url 保存

必要な Supabase Secrets

シークレット名	値
<code>GOOGLE_SERVICE_ACCOUNT_JSON</code>	Googleサービスアカウント JSON (1行に圧縮)
<code>BOX_SPREADSHEET_ID</code>	キュー用スプレッドシートのID

【人間】手作業

1. Googleサービスアカウント作成 (所要: 約20分)

- Google Cloud Console → IAM と管理 → サービスアカウント → 新規作成
- Google Sheets API と Google Drive API を有効化
- キー (JSON) をダウンロード → 1行に圧縮して Supabase Secrets の `GOOGLE_SERVICE_ACCOUNT_JSON` に登録
- box-downloader の `auth/service-account.json` にも同じJSONを設置

2. スプレッドシート作成・共有 (所要: 約5分)

- Googleスプレッドシートを新規作成
- 1行目にヘッダー: `boxurl / driveurl / 実施`
- サービスアカウントのメールアドレスを「編集者」として共有
- スプレッドシートのIDを Supabase Secrets の `BOX_SPREADSHEET_ID` に登録

3. SupabaseでSQL実行 (所要: 約5分)

- `supabase/migrations/add_box_columns.sql` をSQL Editorで実行
- `supabase/migrations/add_enrich_cron.sql` の `YOUR_PROJECT_REF` と `YOUR_SERVICE_ROLE_KEY` を書き換えてから実行

4. box-downloaderのセットアップ (所要: 約15分)

- `setup.bat` を実行して依存パッケージをインストール
- `setup-drive-auth.bat` を実行してGoogleドライブOAuth2認証を完了

- `config.json` にスプレッドシートURL・ドライブフォルダIDを設定
- `run.bat` で動作確認

【Phase 4.8】 skill_master スキルマスター (実装完了・人間手作業待ち)

概要

`skill_master` テーブルにITスキルを蓄積し、`inbound-email` Edge Function で AI を使わずにスキルを照合・抽出する。AI が新スキルを発見した際は自動的に DB に登録し、毎日クリーンアップ Cron でゴミエントリを除去する。

フロー (コミット `139a4f2` で AI 完全廃止後の現行)

- ① inbound-email でメール受信
↓
- ② URL ストリッピング + 送信者署名除去 (コミット ``ccc82ec``)
 - "https://...cc.php" 等が PHP/HTTPS に誤マッチするのを防止
 - "〒XXX-XXXX 東京都..." 等の署名行を都道府県判定から除外
↓
- ③ skill_master DB照合 (AIなし)
 - 本文と添付を別ロジックで照合 (本文: certContext 内のみ資格判定 / 添付: 全文 fallback)
 - スキルシート形式の場合は A/B/C 評価のみ採用、D/E は除外
 - 添付は上位 20 件に絞る (スキルシート一覧の過剰ヒット防止)
 - フェーズ表ヘッダー行 ("調査分析 要件定義 ...") は役割・業界の判定から除外
↓
- ④ regex / 文章スキャンで残りフィールドを抽出 (AIなし)
 - 氏名・最寄駅・都道府県・経験年数・希望単価・参画時期・希望案件
 - 駅 → 都道府県マッピングで送信者署名由来の誤判定を上書き
↓
- ⑤ DB照合スキルを candidates.skills と candidate_skills に保存
 - match_count を RPC でインクリメント (fire and forget)
↓
- ⑥ 毎日 JST 3:00 に skill-master-cleanup が実行
 - ルールベースでゴミエントリ (source='ai') を削除
 - 30日間未マッチ (match_count=0) のエントリを削除

※ ⑤ で `skill_master` に **未登録の業界標準スキル** (JP1/Teraterm/Zabbix/Hinemos/Tivoli/HULFT/上級情報処理士/ITパスポート ほか) はコミット `acf9d31` で 32 件追加済み
(`supabase/migrations/20260520121447_fix_skill_master_quality.sql`)。AI が新スキルを自動登録する経路は inbound-email の AI 廃止により **現状動いていない**点に注意。

新規ファイル一覧

ファイル	内容
<code>supabase/migrations/add_skill_master.sql</code>	skill_master テーブル作成 + increment_skill_match_counts RPC
<code>supabase/migrations/seed_skill_master.sql</code>	~1600件のシードデータ

ファイル	内容
supabase/functions/skill-master-cleanup/index.ts	毎日クリーンアップ Edge Function（AIなし・ルールベース）
supabase/migrations/add_skill_cleanup_cron.sql	クリーンアップ pg_cron スケジュール
scripts/skill_master_review.py	月次レビュースクリプト（Claude Code が毎月実行）

品質チェック（Claude Code が定期実行すること）

「品質チェックして」または `/quality-check` で実行。詳細手順は `.claude/commands/quality-check.md` を参照。

概要:

- 1. **skill_master** メンテ — 不要エントリ削除・未登録スキルの追加
- 2. 駅名マッピング — `[station_unmapped]` ログ確認・頻出駅を追記
- 3. 取りこぼし調査 — 名前不明・null項目・誤登録の原因調査 → 確認後に修正・デプロイ
- 4. 異常監視 — `ai_logs` のエラー率・処理時間を確認

【人間】手作業

- 1. **SupabaseでSQL実行**（所要: 約5分）
 - `supabase/migrations/add_skill_master.sql` を SQL Editor で実行
 - `supabase/migrations/seed_skill_master.sql` を SQL Editor で実行
 - `supabase/migrations/add_skill_cleanup_cron.sql` の `YOUR_PROJECT_REF` と `YOUR_SERVICE_ROLE_KEY` を書き換えてから実行

【Phase 4.9】inbound-email から AI 解析を完全除去（コミット `139a4f2` で完了 + `a4dc3b4` でデッドコードも削除）

背景・成果

- 無料枠の Groq `llama-3.1-8b-instant` が 1 日 125 件で枯渇し無限リトライループに陥っていた
- 名前抽出など regex / 文章スキャン側のフォールバックロジックが十分に成熟していたため AI を排除する判断
- メール取り込みが**完全無料・上限なし**で永続稼働するようになった

現行の抽出パイプライン（AI 不使用）

ステップ	関数	内容
0	<code>TRAINING_REPORT / PROJECT_SOLICITATION</code> フィルタ	「研修内容について報告します」「案件情報のご紹介でございます」等のキーワードを含む人材メールを即スキップ（コミット <code>1631a32</code> ・人材メールボックスへの誤投函対策）
1	<code>decodeHtmlEntities</code>	<code>&amp;</code> 等の HTML エンティティを実体に戻す

ステップ	関数	内容
2	<code>stripUrlsForSkillMatching</code>	URL を除去 (<code>https://.../cc.php</code> 等が PHP/HTTPS に誤マッチするのを防止)
3	<code>stripSenderSignature</code>	「——」 「——」 等の長い区切り線以降を送信者署名とみなして除去
4	<code>extractAndRemoveSkills</code>	<code>skill_master</code> で本文・添付を別ロジックで照合。スペースなし比較 (<code>Spring Boot</code> ↔ <code>Springboot</code>) に対応
5	<code>filterBySkillRating</code>	スキルシートの A~E 評価で D/E 評価のスキルを除外
6	<code>extractCandidateFieldsRegex + flexLabel</code>	氏名・最寄駅・都道府県・経験年数・希望単価・参画時期・希望案件を 2 段階 regex で抽出。 <code>flexLabel</code> でラベル文字間の全角/半角スペースを許容 (単 価 / 氏 名 等)、SEP に】を含めて【単 価】65万 のような囲み記号にも対応
7	<code>inferPrefectureFromStation</code>	駅名から都道府県を逆引きして送信者住所由来の誤判定を上書き。約 254 駅・32 都道府県をカバー (後述)。マップ未収載の駅は <code>console.log('[station_unmapped]', ...)</code> でログ出力
8	<code>isPhaseTableHeader + extractFromProse</code>	フェーズ表ヘッダー行を除外した上で <code>PROSE_ROLES / PROSE_INDUSTRIES</code> を文章スキャン
9	<code>splitMultiCandidateBody</code>	区切り線 (*****/——) で 1 メール=複数候補者を分割 (区切り線 2 本以上を条件・コミット <code>baac676</code> で強化)
10	<code>extractAgentComment</code>	エージェント所感・推薦コメント・備考等を最大 500 字で抽出して <code>raw_profile.agentComment</code> に保存
11	重複判定 (ルールベース)	名前完全一致 + スキル Jaccard ≥ 0.4 → <code>duplicate_flag=true</code> 。駅違いは別人とみなす (コミット <code>0998d49</code>)
12	自動マッチ (任意)	<code>AUTO_MATCH_ENABLED='true'</code> のとき <code>matchCandidateToProject</code> 経由で即時スコア計算 (唯一の inbound-email 内 AI 呼び出し)

削除済みのデッドコード (コミット `a4dc3b4` で全廃)

- `classifyInboundRelevance` (STEP1 関連性チェック)
- `generateJSONSmart, generateJSONWithCerebras, generateJSONWithGroq, generateJSON (kind='candidate'/'project')`
- `buildCandidateGroqPrompt / buildProjectGroqPrompt`
- inbound-email 用の Gemini プロンプトビルダー全般

→ Grep で `supabase/functions/` 全体で 0 ヒット確認済み。残るのは `matchCandidateToProject` 内の `generateJSON kind='match'` のみ。

案件メールの解析（コミット c8be840 で人材と同じ regex 基盤に統一）

フィールド抽出（extractFieldTwoPhase を案件にも適用）

フィールド	ラベル	ISO 変換	特記
場所	場所/勤務地/就業場所/作業場所/常駐先/Working Location	—	駅名のみなら inferPrefectureFromStation で都道府県付与・未解決は [station_unmapped] ログ
単価	50～80万/60万/単金 系	—	WS = '[\\t\\u3000]*' で全角スペース対応
時期	参画時期/開始時期/開始日/期間/稼働開始/契約期間/Period 等	7月～2027年2月 等の範囲を ISO 化	コミット c779a6c で実装
備考	備考/補足	—	コミット 2c73a40 で【内容】 セクションが既にある場合も常に追記
募集人数 / 契約形態 / クライアント / 商流 / 精算幅 / 面談形式	各種	—	すべて extractFieldTwoPhase 経由

ブラケット・デコレータ正規化

- 【場所】 【単価】 【時期】 【備考】 ブラケット形式（コミット 2a43bf5）：SEP 正規表現に 】 を含めることで囲みラベル直後の値を抽出
- ◆氏名◆ のようなデコレータ（コミット 932ce3a）：DEC0_RE で削ってから flexLabel に渡す

スキル抽出の堅牢化

- PROJECT_PROCESS_NOISE = ['システム開発', '機能追加', '改修']（コミット 7d7ff91 で縮小。以前は工程語を多数除外していたが、skill_master に テスト/保守開発/保守運用/調査分析 等を追加して残す方針に転換）
- 【スキル】～次の【...】 セクション内に絞り込んで照合（コミット 95ef77e）
- <尚可> セクション分離で必須スキル / 歓迎スキルを別格納
- SKILL_NOISE_WORDS（必須・歓迎・尚可・優遇・経験・実務・業務・対応・作業・設計・開発 等 23 語）で汎用語を除外

駅 → 都道府県マッピング（コミット 2e9b559 / d7157b7 で大幅拡充）

- 約 254 駅・32 都道府県をカバー（千葉 28・埼玉 19・神奈川 24・東京 42・茨城 6・大阪 29・京都 11・兵庫 15・愛知 17・福岡 17 ほか）
- 案件側にも適用（d7157b7）：勤務地が駅名の場合に都道府県を推定
- 同名駅の衝突（既知の改善余地）：町田（神奈川/東京）・野田（千葉/大阪）・福島（大阪/福島）は後勝ちで上書きされる。配列値化やキー接尾辞化が将来課題

- 未解決駅の運用：`console.log('[station_unmapped]', station)` で Supabase Logs に蓄積 → `.claude/skills/quality-check/SKILL.md` の手順で月次レビュー → `STATION_TO_PREFECTURE` に追記 → `npm run deploy:edge`

【Phase 4.10】 マッチングの全面再設計（コミット `b35df40` で完了）

背景・成果

- AI 呼び出しが 1 ペアごとに発生していたため、案件 × 候補者 = 数百～数千ペアで Groq / Gemini を消費しすぎていた
- 「ルールベース事前フィルタ + バッチ AI 採点」方式に転換し、**1 案件 = 1 AI コール**まで圧縮
- AI が全段失敗してもルールスコアで全代替できるため、無料枠超過時も処理は継続

新 Edge Function `match-batch` (`supabase/functions/match-batch/index.ts`)

モード

- `project_to_candidates`：1 案件 × 多人材を一括採点
- `candidate_to_projects`：1 人材 × 多案件をコメントのみ生成（score は AI に出させない設計）

ルールベーススコア `calcRuleScore` (0～100pt)

観点	配点	ロジック
スキル一致	最大 40pt	<code>required_skills</code> がある場合のみ算出。完全一致 1pt / includes 部分一致 0.5pt → $(hits/required.length) * 40$ 。required 空のときは固定 +20
経験年数	最大 15pt	10年=15 / 7年=12 / 5年=8 / 3年=4 / 1年=2
単価	最大 15pt	<code>budgetMax==null</code> なら +15 固定、範囲内 +15、上限+10% +8、上限+20% +3
勤務地	最大 20pt	同じ都道府県（接尾辞除去 includes 一致） +20、フルリモート（ <code>/フルリモート 完全リモート 100[%]リモート/</code> ） +20、居住地不明 +5
リモート	最大 10pt	<code>!isFullRemote && remoteAvailable && /リモート remote 在宅/i.test(remotePolicy)</code> で +10

バッチ AI プロンプト

- 1 コールで topN 名（既定 10）を一括採点
- 各候補者に `ruleScore` を埋め込み、**「ruleScore を参考にしつつ役割・経歴・希望職種で再採点」** を AI に指示
- `summary` は **150 字以内**で「必須スキル合致 → 経験年数 → 単価 → 勤務地リモート → 懸念点」の優先順
- 出力形式: `[{"id":"...", "score":整数, "summary":"150字以内"}, ...]`

AI フォールバック順

Cerebras `llama3.1-8b` (タイムアウト 20s) → Groq `llama-3.3-70b-versatile` (25s) → Gemini `gemini-2.5-flash` (30s)

失敗時挙動

3 段すべて失敗時は `usedModel='rule'` を返し、ルールスコアで全代替。`results` には topN、`ruleOnly` には残り (AI summary 空) が入る。

match-score (既存・UI 手動・単発スコア)

- 1 ペアの詳細スコア + `duplicateSuspected` フラグ (同名同スキル候補の重複疑い検出)
- マッチング理由 **150 字** (コミット `0d1af7e` で 100 字 → 150 字)
- AI フォールバック順は `match-batch` と同じ
- 居住地・希望勤務地・案件備考・本人希望のスコア反映は `match-batch` の `calcRuleScore` に集約 (`match-score` 側は AI へ丸投げ)

auto-match (毎朝 JST 9:00 cron・コミット `aa480b8` 以降全面書き直し)

- `app_config.auto_match_enabled='false'` でスキップ (既定 true)
- 対象: `data_env='prod'` で `created_at >= NOW() - 25h` の案件
- 既存 `submissions` ペアと `accepted` 状態の人材を除外
- JS 側スキル重複フィルタ (jsonb skills 列に `&&` が使えないため includes でゆるい一致)
- `MAX_CANDIDATES_PER_PROJECT=40 / BATCH_AI_SIZE=20` (20 名 × 2 リクエストで 40 名カバー)
- `match-batch` を `Promise.allSettled` で叩いて `submissions upsert` (`onConflict: 'candidate_id,project_id'`、`ai_raw: { autoMatched: true, source: 'auto-match-cron' }`)

MatchingPage (コミット群: `1bf49ff`, `51f966d`, `aa480b8`, `c6ced01` 等)

- `duplicate_flag=true` と `merged_into != null` の人材をマッチング対象から完全除外
- 候補者取得を RPC 化: `fetch_candidates_for_matching(p_data_env, p_limit DEFAULT 800)` で `created_at DESC`, `COALESCE(experience_years, 0) DESC` 順
- 案件→人材は SQL 側スキル絞り込み: `fetch_candidates_for_project(p_data_env, p_skills text[], p_limit DEFAULT 500)` で `jsonb_array_elements_text` 展開後マッチ
- マッチング詳細パネルに案件サマリー (必須スキル上位 10 件、予算、勤務地、リモート、開始日、roleSummary / description 先頭 150 字)
- bulk マッチング進捗表示 (`MatchRunProgress`) とキャンセル機構 (`bulkCancelRequestedRef`)
- 全 mutation の `onError` で `logError(e, 'MatchingPage', undefined, { dataEnv, nickname })` を呼び `error_logs` に保存

【Phase 4.11】UI 統一とデモ生成のルールベース化 (コミット群: `f28ec86 / 04f0e98 / 3ec217b / 060f0d7 / adc6f3a` ほか)

CandidatePage / ProjectPage

- 「AI で登録」ボタンを廃止し**「登録」ボタンに一本化** (`f28ec86`)
- 登録ボタンは `inbound-email` を `force=true` で叩く (`DEDUP / SENDER_DAILY_LIMIT / inbound_project_enabled` ゲートをバイパス)
- AI なしで登録モード (`04f0e98`): メール取り込みと同じ `regex + skill_master` 方式を直接適用

- 「再解析」ボタン (7a9bb10) : raw_profile.text を本文として inbound-email に再投入（新規 INSERT として）
- 検索スコープ選択 tags / body / all (c5bfd41) : search_candidates(p_scope) の 3 モード
- スキル本人強調度順ソート (1c04c4c) : 出現回数 + 「希望/得意/専門/強み/メイン」等の近傍 ±30 字 +2 + 前半 500 字内 +1
- 返信ボタンに元メール本文引用 (c915e76) : mailto: の body に元差出人・件名・受信日時・本文先頭 800 字
- データ再読み込みボタン (7f4c8ed) : TanStack Query を invalidate
- エージェントコメント表示 (bcd0fdb) : raw_profile.agentComment を黄色枠で whitespace-pre-wrap
- 年齢・性別表示: 経験X年 / X歳（男性/女性）
- 画像アップロードは現状エラー表示（コード残るが UI から呼ばれない）

デモ生成（AI 不使用・ルールベース化）

- DemoSeedPanel (3ec217b で AI 完全除去) : 1 ペア（人材+案件）のテンプレ生成・本番→デモコピー（random/recent モード）・デモメール再解析
- DemoProjectCandidateGen (060f0d7) : 選択中案件ベースのスコア別 5 人生成（90/70/50/30/10pt 想定）
- リアルなエージェントメール本文生成 (0e963d9) : inbound-email regex で確実に拾えるフォーマット
- 表示条件は demoUiEnabled === true のみ（dataEnv 不問・コミット 95120c1）
- 本番→デモコピー（copyProdCandidatesToDemo）: email を demo.prod+<uuid>@demo.invalid に差し替え、resume_url / box_url を落として保存

【Phase 4.12】人材マップ（ヒートマップ）と 7 日アーカイブ機構（コミット群: 2026-05-23）

背景

- 「どの都道府県に何人いるか」を視覚的に把握したいという営業要望
- 単純集計だと過去データ（7 日で削除される人材）が見えないため、アーカイブ機構を併設

新規ファイル

ファイル	内容
src/pages/HeatmapPage.tsx	5 タブ目「人材マップ」。d3-geo (Mercator) + 生 SVG <path> + topojson-client で日本地図描画
src/lib/db/heatmap.ts	RPC ラップ (fetchPrefectureCounts / fetchCandidatesByPrefecture / fetchSkillNames)
public/japan.topojson	47 都道府県の TopoJSON (約 416 KB・properties.nam_ja 使用)
supabase/functions/archive-candidates/index.ts	7 日経過した prod 人材を candidates_archive_light に

ファイル	内容
	サマリー化してから DB 削除
supabase/migrations/20260523_prefecture_counts_rpc.sql	初版 RPC
supabase/migrations/20260523_archive_light_table.sql	candidates_archive_light テーブル + 期間対応版 RPC
supabase/migrations/20260523_normalize_prefecture.sql	normalize_prefecture 関数 + 最終版 RPC prefecture_counts / candidates_by_prefecture
supabase/migrations/add_archive_candidates_cron.sql	旧 delete-old-candidates を unschedule → 新 archive-candidates-daily (毎日 JST 0:00) を schedule

改修ファイル

- src/components/Layout.tsx / src/App.tsx: 5 タブ目「人材マップ」追加 (Map アイコン)
- package.json: d3-geo / topojson-client / topojson-specification / @types/topojson-client / react-simple-maps / @types/topojson-client を追加

主要機能

- 期間トグル: 「直近 7 日」(既定) = candidates のみ / 「全期間」 = candidates_archive_light も合算
- スキルフィルター: skill_master から match_count 降順 200 件をオートコンプリート → cs.skill ILIKE '%skill%' で部分一致
- 塗り色: sqrt(count/max) ベースの薄青～濃青グラデーション。選択中はオレンジ
- 詳細パネル: 都道府県クリックで candidates_by_prefecture RPC を呼び、最大 10 件を created_at DESC で表示 (merged_into IS NULL かつ duplicate_flag = false)。アーカイブ済みは「アーカイブ」バッジ
- キャッシュ: TanStack Query の staleTime で 60s / 5min / 30s

normalize_prefecture 関数

表記ゆれを吸収して都道府県名を正規化:

- '日本' / '関東' / '全国' / 'リモート' / 英字含む → NULL
- '東京都 大森' → '東京都' (regex で切り出し)
- '茨城' → '茨城県' (39 県分の固定リストで接尾辞補完)
- '東京' → '東京都'、'大阪' → '大阪府'、'京都' → '京都府'、'北海' → '北海道'

既知の migration 漏れ

20260523_archive_light_table.sql は name / subject カラムを定義していないが、archive-candidates Edge Function と candidates_by_prefecture RPC が両カラムを参照する。新規環境では以下の手動 ALTER が必要:


```
ALTER TABLE candidates_archive_light
  ADD COLUMN IF NOT EXISTS name text,
  ADD COLUMN IF NOT EXISTS subject text;
```

詳細は docs/Heatmap.md を参照。

【Phase 5】 最終納品ドキュメント作成（一部進行中）

- 1. [Claude] 作業: システム構成図のメンテナンス（README.md に Mermaid 図あり・コミット 2026-05-20 で更新）
- 2. [Claude] 作業: 操作マニュアルのメンテナンス（docs/Sales_Manual.md / docs/Sales_Manual.pdf ・営業担当者向け）
- 3. [Claude] 完了: 全成果物を commit & push し、納品完了

4. Claude Codeへの重要な行動指針

- 正の所在: 仕様・挙動の優先順位は 本リポジトリのソース と README.md。本ファイル（CLAUDE.md）はそれに追従するメモであり、食い違いがあれば ソースを正として本ファイルを更新すること。
- こまめな Git 操作: 機能実装単位、またはテスト通過ごとに、意味のあるメッセージと共に commit & push を行うこと。
- バグゼロの追求: ロジックには必ずテストコードを付随させ、テスト項目書をエビデンスとして出力すること。
- ドキュメントの対象読者:
 - README/構成図は「後任エンジニア」が最短で再現できるように。
 - 操作マニュアルは「非IT営業職」がIT用語なしで理解できるように。

5. データベース構成

candidate_skills のカテゴリ CHECK 制約は supabase/migrations/add_candidate_skills.sql を正とする。

テーブル一覧

テーブル	用途
candidates	人材マスタ。スキル・経歴・raw_profile を保持。data_env（prod demo）で論理分離。 主要カラム: box_url, box_status, resume_url, drive_url, desired_rate, from_company, duplicate_flag, merged_into
projects	案件マスタ。必要スキル・予算・raw_data を保持。data_env 同上
submissions	マッチング提案履歴。スコア・AI要約を保持。data_env 同上
candidate_skills	スキルをカテゴリ別に分解して保持（検索最適化・14カテゴリ）
candidates_archive_light	人材マップ用サマリーテーブル（Phase 4.12）。7 日経過した prod 人材の id/data_env/prefecture/skills/created_at/archived_at +

テーブル	用途
	<code>name/subject</code> を保持。 <code>archive-candidates</code> Edge Function が毎日 JST 0:00 に upsert + 元データ削除
<code>ai_logs</code>	AI解析の実行ログ（モデル・所要時間・結果・エラー）。メール解析の AI 廃止後、 <code>inbound-email</code> 由来のレコードは <code>model='no-ai'</code> で保存される
<code>error_logs</code>	フロントエンド側のクライアントエラーを記録（コミット <code>a2c0e96</code> ）。 <code>page/message/stack/context/data_env/nickname</code> を保持。 <code>saveErrorLog/logError</code> ユーティリティから呼び出し。30 日自動削除 cron は未実装（要追加）
<code>skill_master</code>	ITスキルマスタ。約 1,660 件規模（ <code>acf9d31</code> で +32 件、Phase 4.10 で DWH/工程/IBM 系を +32 件追加）。 <code>aliases</code> で表記ゆれ吸収・ <code>match_count</code> / <code>last_matched_at</code> でマッチ実績管理
<code>relevance_keywords</code>	関連性キーワード辞書（ <code>exclude</code> / <code>candidate</code> / <code>project</code> の 3 種別）。 <code>classifyInboundRelevance</code> で使用予定だったがコミット <code>a4dc3b4</code> で関数自体が削除されたため、現状の <code>inbound-email</code> 経路では未使用
<code>app_config</code>	アプリ全体設定。Microsoft OAuth リフレッシュトークンのローテーション保存・各種機能フラグも保持（後述）

app_config の主要キー

キー	既定	内容
<code>inbound_project_enabled</code>	<code>false</code>	案件メールの解析と DB 保存を有効化
<code>auto_match_enabled</code>	<code>true</code>	<code>auto-match</code> cron を有効化。 <code>'false'</code> でスキップ
<code>email_poll_mode</code>	<code>incremental</code>	<code>incremental</code> （未読のみ）か <code>full</code> （指定日以降全件）
<code>email_full_import_since</code>	（未設定）	<code>full</code> モード時の開始 ISO 日時
<code>email_classify_enabled</code>	<code>false</code>	<code>poll-email</code> 内の Gemini メール種別分類
<code>matching_fast_max_candidates</code>	<code>20</code>	高速モード時の案件あたり候補者上限
<code>matching_fast_max_projects</code>	<code>10</code>	高速モード時の人材あたり案件上限
<code>candidate_retention_days</code>	<code>7</code>	人材データ保持日数（旧データ自動削除用・運用判断で活用）
<code>app_memo</code>	（未設定）	営業引き継ぎ用フリーテキストメモ
<code>graph_rt_human_prod</code> ほか	—	Microsoft OAuth 連携で保存されるリフレッシュトークン（4 アカウント分）

candidate_skills の14カテゴリ

カテゴリ	内容
languages	プログラミング言語・クエリ言語
frameworks	フレームワーク
libraries	ライブラリ、UIキット等
os	OS
databases	RDB・NoSQL・KVS
dwh	DWH・BI 等
clouds	クラウドサービス
infrastructures	インフラ技術（コンテナ・IaC 等）
tools	Git, Jira, Slack, Notion, BIツール等
methodologies	PM・マネジメント系
certifications	資格試験等
design	デザイン・クリエイティブ系
marketing	マーケティング・集客系
others	その他

6. 実装済み機能の詳細

メール自動受信（現行・ポーリング方式）

poll-email と inbound-email の役割分担

pg_cron（5分ごと）

↓

【poll-email】メール取得係

- Microsoft Graph API で Outlook の未読メールを取得
- AI種別判断（有効時）： candidate / project / other を判定
- 処理済みメールを既読マーク（重複防止）
- メール内容を inbound-email に HTTP POST で渡す（1件ずつ）

↓

【inbound-email】解析・保存係（コミット `139a4f2` で AI 廃止・`a4dc3b4` でデッドコード全削除・`c8be840` で案件解析統一）

- STEP0-2： メタ情報・本文・添付の受け取りと検証
- STEP3： Word/Excel 添付をテキスト変換（PDF は Storage 保存のみ・解析しない）
- STEP4： メール本文中の Google Drive / Sheets / Docs リンクを取得
- STEP5： ★ AI 廃止 ★ regex + 文章スキャン + skill_master DB 照合で構造化抽出
 - 人材経路： HTMLエンティティ復号 → URL除去 → 署名除去 → skill_master照合 →

→

extractCandidateFieldsRegex（flexLabel+SEP=】対応）→ 駅→都道府県

→

extractFromProse（フェーズ表ヘッダー除外）→

splitMultiCandidateBody → 重複判定

– 案件経路: 同じ前処理 → extractFieldTwoPhase (場所/単価/時期/備考/募集人数等) →

【内容】セクション抽出 → 【備考】常時追記 → 駅→都道府県(d7157b7) → スキル抽出 (尚可セクション分離)

– STEP6-7: 解析結果を candidates / projects テーブルに DB 保存 (ai_logs.model='no-ai')

– STEP8: 任意・AUTO_MATCH_ENABLED='true' のとき matchCandidateToProject 経由で即時マッチ

– 案件メール処理は app_config.inbound_project_enabled='true' のとき**のみ**実行 (既定 OFF)

– 手入力登録ボタンは force=true で DEDUP/SENDER_DAILY_LIMIT/inbound_project_enabled をバイパス

ポイント: inbound-email は今も現役。poll-email は「Outlookからメールを取ってきて inbound-email に渡す橋渡し役」。Make.com が廃止された後も inbound-email の枠組みはそのまま流用しているが、AI 呼び出しは完全に除去され関数定義レベルでも残っていない。

- **Edge Function:** supabase/functions/poll-email/index.ts (Phase 4.5 で実装)
- **スケジューラ:** Supabase pg_cron (5分ごとに起動)
- **認証:** Microsoft Graph API (OAuthリフレッシュトークン方式)
- **人材用:** akinavi.hr.ai.voice.human@outlook.jp
- **案件用:** akinavi.hr.ai.voice.project@outlook.jp
- 処理済みメールは既読マークで重複取得を防止

メール設定UI (src/pages/SettingsPage.tsx)

- **設定タブ** をナビゲーションに追加 (src/components/Layout.tsx)
- **メールアドレス設定:** 人材用・案件用アドレスを表示用に app_config へ保存 (参照用。実認証情報は Supabase Secrets)
- **AI種別判断:** 同じ受信箱に人材・案件メールが混在する場合、Gemini AI で candidate / project / other を自動分類
 - 有効時は人材用アカウントのみポーリング (同一受信箱を2重処理しない)
 - other と判断されたメールは既読マークしてスキップ
 - 10秒タイムアウト、失敗時は candidate にフォールバック
- **全件取り込みセクション:** 削除済み (7日以上前のデータは不要のため)
- **DB:** src/lib/db/emailSettings.ts で app_config から設定を読み書き
- **マイグレーション:** supabase/migrations/add_email_settings.sql

全件取り込みモード (poll-email Edge Function)

- **通常モード (incremental) :** 未読メールのみ取得 (通常運用)
- **全件モード (full) :** email_full_import_since 以降の全メールを順次取得 (isRead フィルターなし)
 - 1バッチ最大 50 件 (コミット 2afe469 で 20→50 に拡大)、@odata.nextLink でページネーション継続
 - バッチごとに nextLink を app_config (email_full_import_nextlink_<configKey>) に保存し、5 分ごとに続きから再開
 - 全アカウント完了時に自動で incremental モードに戻す
- **UI:** 設定ページの全件取り込みセクションは **削除済み** (7 日以上前のデータは不要のため運用上使用しない)

- 必要の場合は Supabase SQL Editor で `app_config` の `email_poll_mode` を `full`、`email_full_import_since` を開始日付に手動設定すること

メール自動受信（旧方式・停止中）

- **Make.com:** 無料枠 約1,000ops/月 → 超過により停止
- **Pipedream:** 無料枠 10クレジット/日 → 当日中に超過・停止
- **Power Automate:** 外部URLへのHTTP POSTがプレミアムコネクタのため不採用

Edge Function `inbound-email` (`supabase/functions/inbound-email/index.ts` 準拠)

- **データ環境:** ボディまたはクエリの `mode / data_env` (`prod | demo | dev`)。省略時は `prod`
- **Secrets:** `SUPABASE_URL`、`SUPABASE_SERVICE_ROLE_KEY` (AI 廃止により `GEMINI_API_KEY` 等は基本不要・即時マッチを使う場合のみ追加で `GEMINI_API_KEY / GROQ_API_KEY / CEREBRAS_API_KEY`)
- **app_config フラグ:** `inbound_project_enabled='true'` を設定するまで案件メールは解析せずスキップする (既定 OFF)
- **AUTO_MATCH_ENABLED** (env): `true` で `matchCandidateToProject` 経由の即時マッチを有効化 (既定 false)
- **force=true** (body): `DEDUP / SENDER_DAILY_LIMIT / inbound_project_enabled` ゲートをバイパス。手入力登録ボタン経由でこのフラグが付く
- **INBOUND_MAKE_SOFT_FAIL:** 例外時も HTTP 200 + `ok:false` で返す (外部サービス停止回避用に名残として残置)
- 本文・添付とも空: HTTP 200 + `skipped`

論理データ環境 `data_env`

- `prod / demo` を同一Supabase内で分離 (`data_env` カラムでフィルター)
- **デモ解除:** `VITE_DEMO_KEY` と URL クエリ `?demo=<鍵>` でトグル
- 解除時にヘッダの「データ」セレクトが表示される
- デモ UI 未解除時は常に `prod` 固定

マッチング画面（Phase 4.10 で全面再設計）

マッチング方式の比較と使い分け

	高速モード (fast)	全件モード (full)	自動バッチ (daily cron)
実行 タイ ミン グ	手動 (UIボタン)	手動 (UI ボタン)	毎朝9時 自動
案件 →人 材	上限 <code>matching_fast_max_candidates</code> (既定 20)	全候補者	<code>MAX_CANDIDATES_PER_PROJECT=40</code>
人材 →案 件	上限 <code>matching_fast_max_projects</code> (既定 10)	全案件	未対応

	高速モード (fast)	全件モード (full)	自動バッチ (daily cron)
AI 呼び出し方式	match-batch (1 案件 1 コール・topN を一括採点)	同上	同上
AI フォールバック	Cerebras→Groq 70B→Gemini (3 段全失敗時はルールスコアで全代替)	同上	同上
速度	速い (数秒〜数十秒)	遅い (数分〜)	気にしない (バックグラウンド)
追加実装	fetch_candidates_for_matching RPC + match-batch	同上	auto-match Edge Function + pg_cron

推奨する使い分け:

- 日常運用 → 自動バッチに任せる (毎朝9時に前日登録分が自動でマッチング済み)
- 急ぎで確認したい → 手動・高速モード (数秒〜数十秒で完了)
- 念入りにやりたい → 手動・全件モード (時間がかかるが全候補と照合)

候補者取得の RPC 化 (コミット 51f966d / a2c0e96)

- fetch_candidates_for_matching(p_data_env, p_limit DEFAULT 800):merged_into IS NULL で created_at DESC, COALESCE(experience_years, 0) DESC 順
- fetch_candidates_for_project(p_data_env, p_skills text[], p_limit DEFAULT 500): 案件の必須スキルを SQL に渡して PostgreSQL 側で jsonb_array_elements_text 展開してマッチ。skills (jsonb) 列に && が使えない問題を回避
- MatchingPage は duplicate_flag=true と merged_into != null の人材を完全除外 (コミット 1bf49ff)

match-batch のスコア配点 (ルールベース 100pt)

観点	配点	備考
スキル一致	最大 40pt	required 空のときは固定 +20pt
経験年数	最大 15pt	10年=15 / 7年=12 / 5年=8 / 3年=4 / 1年=2
単価	最大 15pt	予算未設定なら +15、範囲内 +15、上限+10% +8、上限+20% +3
勤務地	最大 20pt	同じ都道府県 / フルリモートで +20、居住地不明 +5
リモート	最大 10pt	リモート可・希望時 +10

AI 採点は **topN 件 (既定 10 件) のみ** バッチプロンプト 1 コールで実施し、残りはルールスコアのみで返す (ruleOnly 配列)。

自動バッチ (supabase/functions/auto-match/index.ts ・コミット aa480b8)

- **スケジュール:** 毎日 0:00 UTC (日本時間 9:00)。app_config.auto_match_enabled='false' でスキップ
- **対象:** 直近 25 時間以内に登録された prod 案件
- **除外:** 既に submissions が存在するペア、accepted ステータスの人材
- **AI 呼び出し:** match-batch 経由 → Cerebras → Groq 70B → Gemini フォールバック
- **マイグレーション:** supabase/migrations/add_auto_match_cron.sql
 - YOUR_PROJECT_REF と YOUR_SERVICE_ROLE_KEY を書き換えてから Supabase SQL Editor で実行すること
- **MatchingPage は常時マウント:** タブ切替で mutation が中断されないよう hidden で切替

デモ生成 (AI 不使用・ルールベース)

DemoSeedPanel (src/components/DemoSeedPanel.tsx・コミット 3ec217b で AI 完全除去)

- **1 ペア生成:** buildDemoPair() がテンプレ + 経験年数からエージェントメール体裁の本文を構築 (inbound-email regex が確実に拾えるフォーマット 氏名 : 最寄駅 : 希望単価 :)
- **本番→デモコピー (random) :** copyProdCandidatesToDemo(count, nickname, 'random')
- **本番→デモコピー (recent) :** recent モードで直近登録分から取得
- **デモ人材メール再解析:** raw_profile.text を持つ人材から count 件をシャッフル選択し inbound-email 再投入
- **表示条件:** demoUiEnabled === true のみで表示 (dataEnv 不問・コミット 95120c1 で prod でも見えるよう変更)
- **email 差し替え規則:** 本番→デモコピー時は demo.prod+<uuid>@demo.invalid に変換、resume_url / box_url を落として drive_url のみ保持

DemoProjectCandidateGen (コミット 060f0d7・新規)

- 選択中案件をベースに **スコア別 5 人 (90/70/50/30/10pt 想定)** を生成
- calcRuleScore の予想値を逆算して必須スキル数・経験年数・単価・勤務地を調整
- 別ドメインスキルを getUnrelatedSkills で混入
- 挿入先は常に dataEnv='demo'

画面構成

- タブは **5 つ (マッチング結果 / 人材登録 / 案件登録 / 設定 / 人材マップ)**。
src/components/Layout.tsx の NAV_ITEMS を正とする
- **設定** タブは Microsoft アカウント連携・案件メール解析の有効化トグル・自動マッチング ON/OFF・人材データ保持日数・マッチング高速モード上限・アプリメモなどをまとめる
- **人材マップ** タブは Phase 4.12 で追加。src/pages/HeatmapPage.tsx で d3-geo + public/japan.topojson を使った日本地図ヒートマップを表示。詳細は docs/Heatmap.md 参照
- **提案履歴 / 重複管理 / 解析監視** は実装済みだがナビから非表示 (src/pages/HistoryPage.tsx, DuplicatePage.tsx, MonitorPage.tsx は存在するが App.tsx 経由で参照されていない)

人材・案件ページの追加機能 (Phase 4.11)

共通

- 「AI で登録」ボタン廃止 → 「登録」ボタン一本化 (コミット f28ec86)
- 登録ボタンは inbound-email を force=true で叩く (DEDUP / SENDER_DAILY_LIMIT / inbound_project_enabled ゲートをバイパス)

- **データ再読み込みボタン** (コミット `7f4c8ed`) : TanStack Query を invalidate して最新化

CandidatePage

- **「再解析」ボタン** (コミット `7a9bb10`) : `raw_profile.text` を本文として `inbound-email` に再投入 (既存候補は残る・新規 INSERT として)
- **検索スコープ選択 tags / body / all** (コミット `c5bfd41`) : `search_candidates(p_scope)` の 3 モード
 - `tags`:
name/skills/desired_rate/from_company/prefecture/nearestStation/currentWorkLocation/summary/agentComment/skillsByCategory/roles/industries の 12 フィールド ILIKE
 - `body`: `raw_profile->>'text'` のみ
 - `all`: 従来動作 (name + skills + raw_profile 全体)
- **スキル本人強調度順ソート** (コミット `1c04c4c`) : 出現回数 + 「希望/得意/専門/強み/メイン/主に/中心/注力/押し/自信」近傍 ±30 字 +2 + 前半 500 字内 +1 で重み付け
- **返信ボタンに元メール本文引用** (コミット `c915e76`) : `mailto:` の body に元差出人・件名・受信日時・本文先頭 800 字
- **エージェントコメント表示** (コミット `bcd0fdb`) : `raw_profile.agentComment` を黄色枠で `whitespace-pre-wrap` (`extractAgentComment` で抽出)
- **年齢・性別表示**: 経験X年 / X歳 (男性/女性)

Google Drive / Sheets / Docs 自動取得

- メール本文中のリンクを自動検出・取得 (`fetchGoogleLinks`)
- Sheets → CSV、Docs → txt、Drive PDF → base64化して `inbound-email` の解析対象に追加
- 認証不要 (リンクを知っている全員が閲覧可の共有設定前提)

ファイルアップロード解析 (ブラウザ)

- **実装**: `src/lib/fileParser.ts`
- **PDF**: 現状未対応。CandidatePage / ProjectPage で「PDF はテキスト解析対象外です。テキストを手動で貼り付けてください」とエラー表示して処理中断。`pdfjs-dist` は package.json に残るが import されていない
- **画像 (JPG/PNG等)**: 現状未対応。`AnalyzeCandidateRequest.imageFiles` のフィールドは型定義に残るが、登録ボタンは `inbound-email` regex 経路に統一されたため UI から呼ばれない
- **Excel (.xlsx/.xls)**: `xlsx` (SheetJS) で全シートを CSV 変換 → テキストエリアへ自動転記
- **Word (.docx)**: `mammoth` で本文テキスト抽出 → テキストエリアへ自動転記
- 対応ページ: 人材登録 (`CandidatePage.tsx`)・案件登録 (`ProjectPage.tsx`)
- 複数ファイル同時選択可。テキスト貼り付けとの併用も可能

AI プロバイダー

- **ブラウザ (マッチング UI など補助用途)**: Gemini (既定 `gemini-2.5-flash-lite`、`VITE_GEMINI_MODEL` で上書き可)。人材・案件登録の UI からは Gemini を呼ばなくなった (Phase 4.11)
- **サーバー (match-batch Edge Function・新方式)**: Cerebras `llama3.1-8b` → Groq `llama-3.3-70b-versatile` → Gemini `gemini-2.5-flash` フォールバック。3 段失敗時はルールスコアで全代替
- **サーバー (match-score Edge Function・単発)**: 同じ 3 段フォールバック
- **サーバー (auto-match Edge Function)**: `match-batch` を内部呼び出しして同じ 3 段フォールバックを継承

- サーバー (**poll-email** メール種別分類) : Gemini **gemini-2.5-flash-lite** バッチ (任意・既定 OFF)
- メール解析 **inbound-email**: AI 不使用 (Phase 4.9)
- フロントは **AIProvider** インターフェースで抽象化 (OpenAI 切替はスタブのみ)

Edge Function デプロイ前検査 (コミット **a8422fb**)

- **scripts/check-and-deploy-edge.sh** : **deno check** を実行し TS2304 (未定義変数) が出たらデプロイを中止
- 背景: コミット **9ef7638** 「fix: sepRe の未定義参照を SEP_LINE_RE に修正」のような事故防止
- **package.json** に **npm run check:edge <function> / npm run deploy:edge <function>** を登録 (既定 **inbound-email**)

品質チェック (**.claude/skills/quality-check/SKILL.md //quality-check** コマンド)

- ① 駅マッピング: [**station_unmapped**] ログを Supabase Dashboard → Functions → inbound-email → Logs から月次レビュー → **STATION_TO_PREFECTURE** に追記 → **npm run deploy:edge**
- ② スキルマスタ: **scripts/skill_master_review.py** で **source='ai'** 怪しいスキルの削除候補 SQL を出力
- ③ 誤登録パターン検出: **TRAINING_REPORT / PROJECT_SOLICITATION** の [**SKIP_IRRELEVANT**] ログを確認
- ④ 重複候補者の手動マージ判定
- ⑤ **AI コスト監視** (コミット **bd13e85**) : **ai_logs** でモデル別・日次呼び出し数を集計 → Gemini 無料枠超過 / プリペイドクレジット枯渇 / フォールバック多発の検知

認証・ニックネーム制

- ログイン機能なし
- 初回アクセス時にニックネームを入力させ **localStorage** に保存

データ重複管理

- email が同じ場合は自動で既存レコードを UPDATE (上書き更新)
- 名前完全一致 + スキル Jaccard ≥ 0.4 のルールベース判定で **duplicate_flag = true** を立てるだけ (自動マージ不可・AI 不使用)
- 駅違いは別人とみなす (コミット **0998d49**)
- マージは UI から手動操作 (**merged_into** カラムに参照先 ID を保存)

AI解析ログ (ai_logs)

- 全 AI 解析呼び出しを DB に記録 (モデル名・所要時間 ms・結果 JSON・エラー)
- 成功・失敗どちらもログに残す
- メール解析 (**inbound-email**) は AI を使わなくなったが、後方互換のためログ自体は引き続き **model='no-ai'** で記録される

クライアントエラーログ (error_logs)

- フロントエンドで捕捉した例外を **saveErrorLog / logError** 経由で **error_logs** テーブルに保存 (コミット **a2c0e96**)
- **page/message/stack/context/data_env/nickname** を記録

- MatchingPage の全 mutation の **onError** で呼び出し → 自動マッチ / バルクマッチ失敗の原因を後追い可能
- 30 日自動削除 cron は未実装（要追加・Phase 5 タスク）